Habib University

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 11 - Part 2

Constant Correctness

THIS WEEK'S AGENDA

01. What is constant correctness?
02. Some constant correctness cases
03. Constant Member Functions

Constant Correctness

WHAT IS CONSTANT CORRECTNESS?

- In C++, constant correctness refers to the use of the **const** keyword to ensure that certain data cannot be altered after its definition.
- This allows us to prevent accidental changes to values which are not meant to be changed.
- It makes the code more predictable and helps to reduce potential bugs.
- Constant correctness is commonly applied to variables, pointers, and member functions of classes.

DECLARING CONST VARIABLES

- Declaring a variable with the const keyword disallows it from being modified after initialization.
- For example:

```
const int byteLength = 8;
/* Gives an error - byteLength is constant
and cannot be changed */
byteLength = 9;
```

DECLARING CONST POINTERS

- In C++, pointers can be made constant in two ways:
  - **Pointer to a constant:** You cannot modify the value being pointed to.

```
const int *ptr = &byteLength;
// ptr points to a constant
int* ptr = 10;  // Error: value cannot be changed
```

  - **Constant pointer:** The pointer cannot change to point to a different address.

```
int value = 5;
int* const ptr = &value;
// error const ptr cannot point to another address
ptr = &anotherValue;
```

COMBINING THE PREVIOUS 2 CASES

- Let's make a constant pointer that points to a constant.

```
const int* const ptr = &byteLength;
// ptr is a constant pointer to a constant int
```

- Neither the address nor the value being pointed to can be changed.

Constant Member Functions

CONSTANT MEMBER FUNCTIONS

- Declaring a member function with const prevents it from modifying any member variables of the class. For example:

```
class Book {
    private:
        int value;
    public:
        int getValue() const {
            return value;
        }
        void setValue(int v) {
            value = v;
        }
};
```

Benefits of constant correctness

BENEFITS OF CONSTANT CORRECTNESS

- Ensures Integrity: Guarantees that certain parts of your program state cannot be altered after being set.
- Enhances Code Clarity: Makes it clear to programmers which values are expected to remain constant.
- Enables Optimization: Allows compilers to perform optimizations since they know that certain values won't change.

Using constant correctness can improve both the reliability and efficiency of your C++ code.

Summary

SUMMARY

- **Virtual functions** are a useful mechanism in C++ to implement polymorphism.
- Virtual functions begin with the keyword **virtual**.
- A derived class is supposed to override a virtual function inherited from the base class to exhibit polymorphic behavior.
- Pure virtual functions do not have any implementation.
- A class containing a pure virtual function is an **abstract class**.

READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 11

ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science